Defining Variables – Overview

In this lesson, we're diving into TypeScript variables, their types, and how to manipulate them. TypeScript offers a variety of basic types including Booleans (`true` or `false`), Numbers (both integers and floating-point numbers), and Strings (text data enclosed in either single or double quotes). In addition to these, TypeScript also supports the `any` type for variable assignments of any data type. For more advanced types, please refer to the official TypeScript documentation.

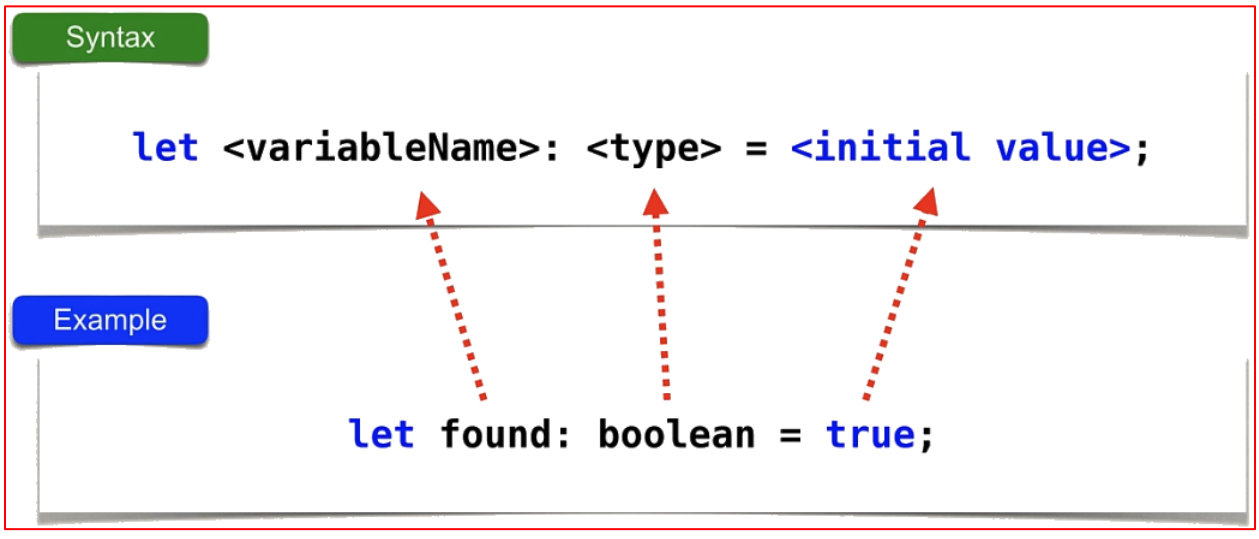
Type	Description
`boolean`	true/false values
`number`	Supports integer and floating point numbers
`string`	Text data. Enclosed in single or double quotes
`any`	Supports "any" datatype assignment
Others ...	See details at www.typescriptlang.org

Variable Declaration in TypeScript

To declare a variable in TypeScript, you use the `let` keyword followed by the variable name, a colon, the type, and optionally an initial value. For example:

```
let found: boolean = true;
```

Here, `found` is the variable name, `boolean` is its type, and `true` is its initial value.



Examples of Variable Declaration

Let's look at some other examples:

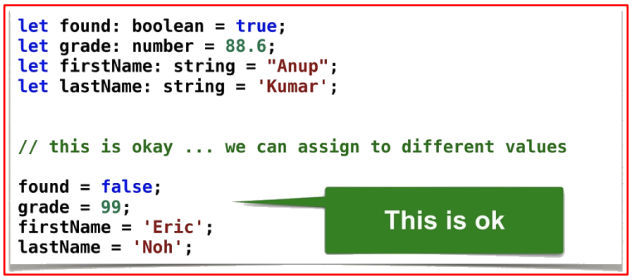
- Boolean: `let found: boolean = true;` (Values can be `true` or `false`)
- Number: `let grade: number = 88.6;` (Values can be integers or decimals)
- String: `let firstName: string = "Anup"; let lastName: string = "Kumar";` (You can use either single or double quotes)

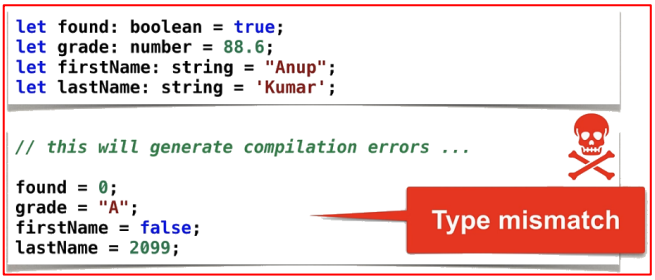
Use `let` Over `var`

We use the `'let'` keyword in TypeScript for variable declaration, in contrast to the traditional JavaScript `'var'` keyword. The `var` keyword comes with several drawbacks such as scoping issues and shadowing, which are mitigated by using `let`.

Strongly Typed Variables

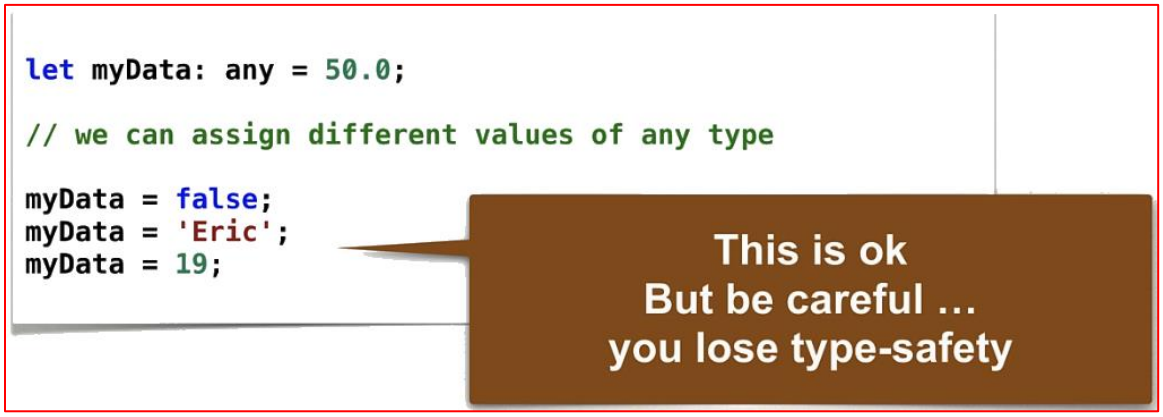
TypeScript is strongly typed, meaning you can only assign values of the type specified during variable declaration. For instance, assigning a different type to a variable will result in a compilation error, which is a feature that enhances code robustness.





Using `any` Type

Although `any` type allows for flexibility, it does compromise type safety. Use it cautiously and primarily for situations requiring generic types, like lists or arrays containing elements of different types.



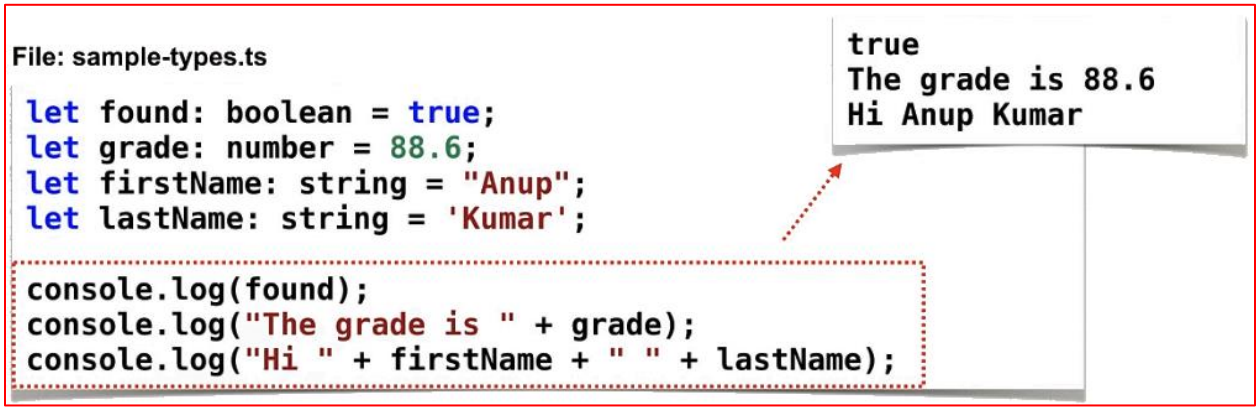
Output Display

To display variable values, `console.log` can be used as follows:

```
console.log("The grade is: " + grade);
```

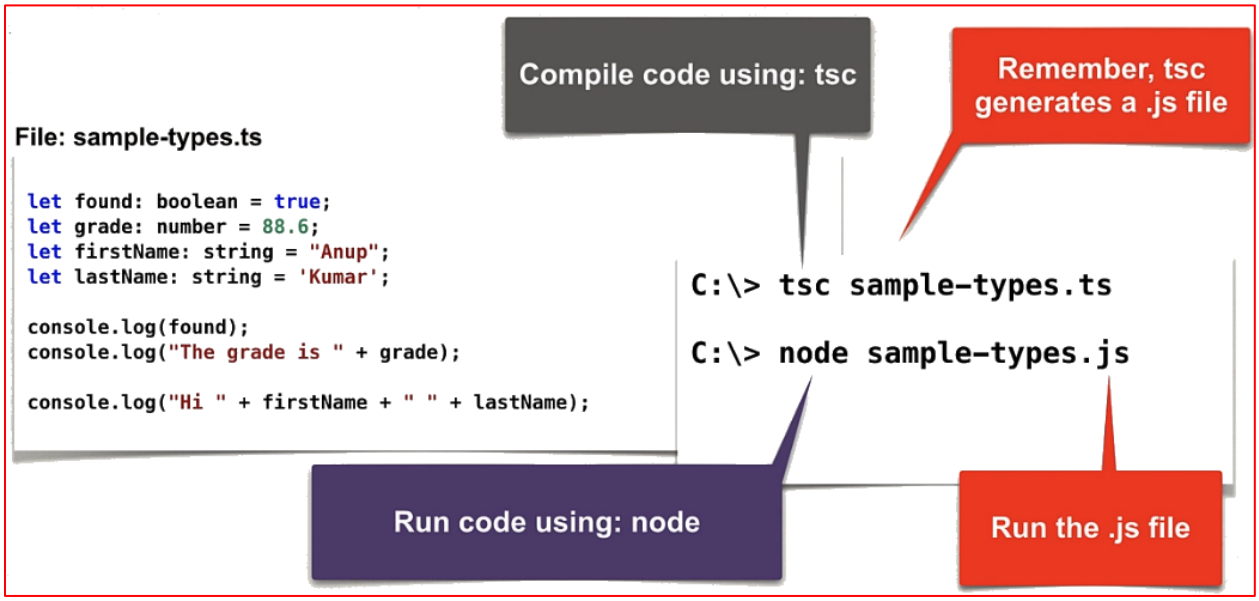
You can also utilize template strings for more complex string concatenation:

```
console.log(`Hi ${firstName} ${lastName}`);
```



Running The TypeScript File

After writing your TypeScript file, you'll need to compile it using the TypeScript Compiler (`tsc`) to generate the corresponding JavaScript file. Then, you can execute the generated file using Node.js.



Common Errors and Troubleshooting

The TypeScript compiler will flag type mismatches and show you the exact lines where errors occur. Remember, even if there are compilation errors, the `tsc` compiler will still generate a JavaScript file by default. To prevent this, use the compiler flag `--noEmitOnError`.

In summary, we've covered TypeScript variables, their types, and nuances such as strong typing and potential pitfalls when not adhering to these types. In the next section, we'll be writing some code inside an IDE to demonstrate these concepts in action.

Defining Variables – Write Some Code

In this section, we'll be diving into the fascinating world of variables in TypeScript. Let's start by creating a new folder within our TypeScript training environment; I'll name it `02_Variables`. Inside this folder, let's also create a new file and call it `sample.ts`.

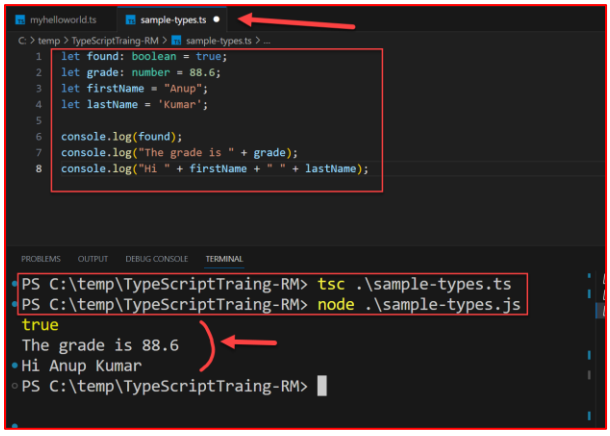
We'll begin by defining some variables in `sample.ts`:

- A boolean variable named `found`, set to `true`.
- A number variable named `Grade`, set to `88.6`.
- String variables for `firstName` and `lastName`, which I'll set to 'Anoop' and 'Kumar', respectively.

Note that you have the flexibility to use either double or single quotes for string variables; it's entirely up to you.

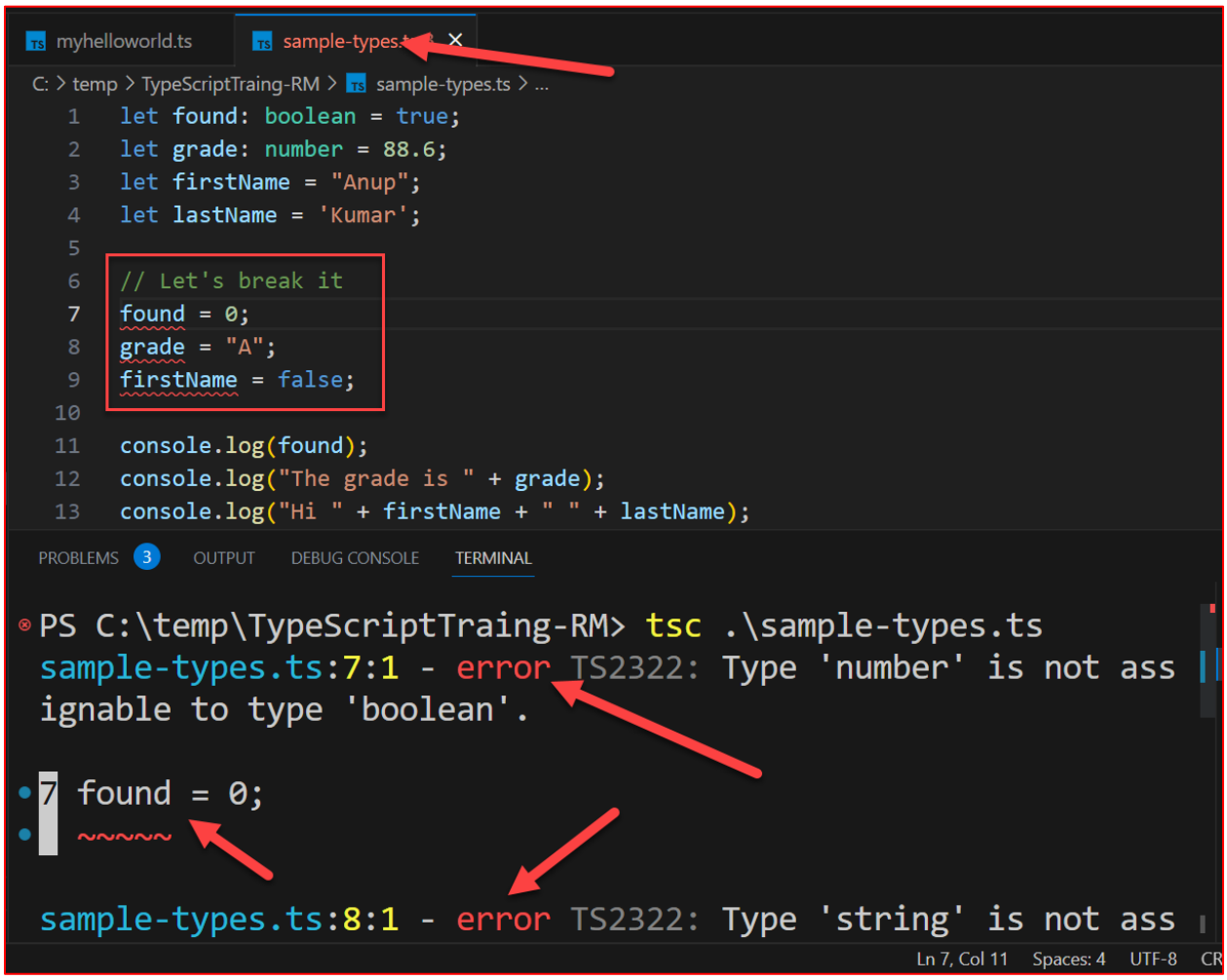
Next, we'll display these variables using `console.log()`. To concatenate the `firstName` and `lastName`, we'll use string concatenation. Here's how that looks: "`Hi, " + firstName + " " + lastName`.

Now, let's run our code. Open a terminal window, navigate to the `02_Variables` directory, and compile the file using the `tsc sample.ts` command. To execute the code, use `node sample.js`. The output will display our defined variables and the concatenated names.



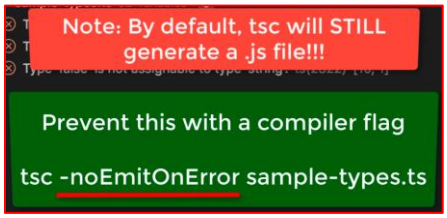
But what happens if we try to "break" our code? For instance, what if we assign a zero to our boolean variable `found`? The IDE will promptly show an error, as it should. A boolean can't be assigned a numerical value.

Similarly, if we try to assign an 'a' to `Grade`, another error will pop up. These errors are also listed in the terminal when we attempt to compile with `tsc sample.ts`, reinforcing the type-checking power of TypeScript.

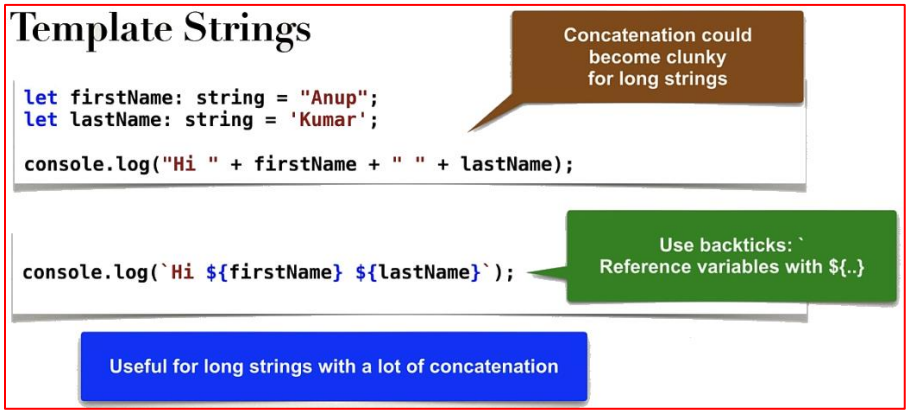


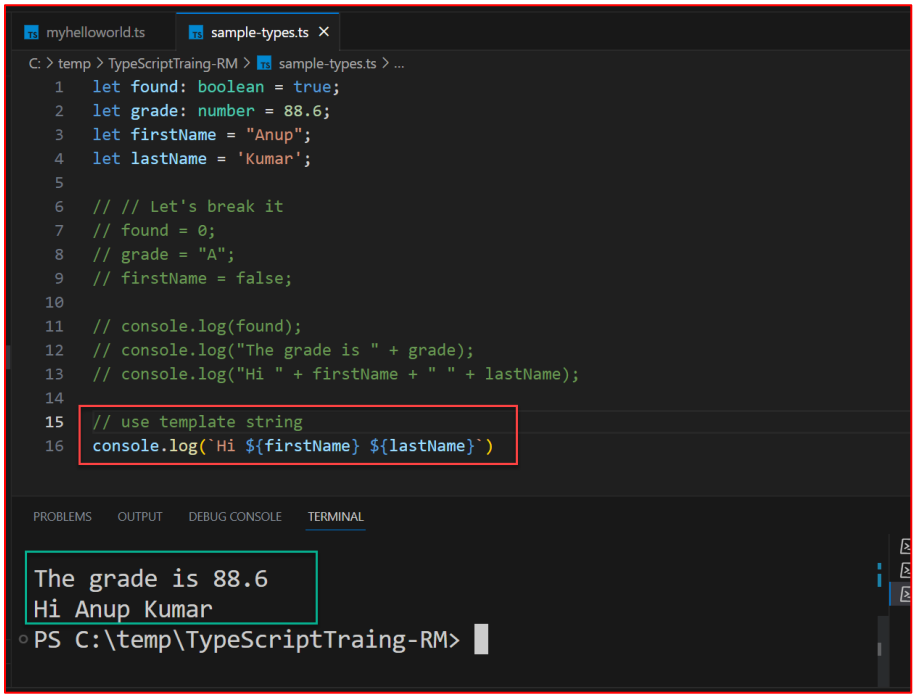
One thing to note: by default, TypeScript will still generate a JavaScript file even when there are errors. To prevent this, add the compiler flag `--noEmitOnError`.

```
tsc -noEmitOnError .\sample-types.ts
```



Now, let's try using template strings for our greeting. With template strings, we can embed variables directly into the string using `${}` syntax, eliminating the need for explicit concatenation. So, `` `Hi ${firstName} ${lastName}` `` will give us the same output.





In summary, we've learned how to define and manipulate variables in TypeScript and also how to display output in various ways.